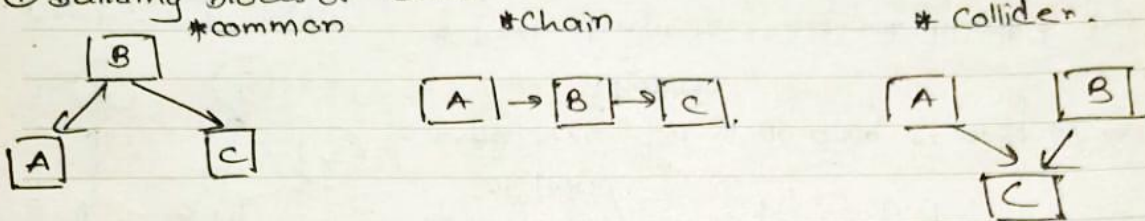


* DAG: Directed Acyclic Graphs.

- Network of nodes connected with ^{directed} edges without loops.
- also used to optimize applications.

① Building blocks of DAG:



* Dynamic programming is a optimization mechanism where it finds all optimum solutions and selects the best one.

* Normally Dynamic Programming problems are structured as DAG.

* Understanding DFS and BFS is important for DAGs.

* Dynamic programming follows principal of optimality. : PO := says that a problem can be solved by taking a sequence of decisions, to get optimal solution.

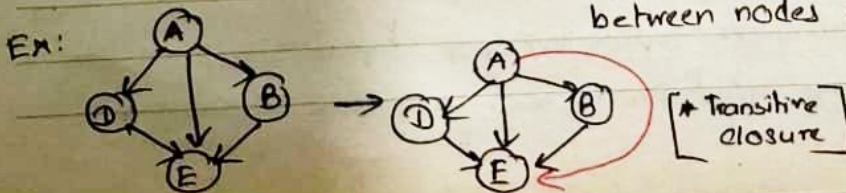
* DAGs are used to show how things are related or dependant on each other, in a clear and organized way.

* DAG $\begin{cases} \text{Directed} \\ \text{Acyclic (no-cycles)} \end{cases}$ + graph.

* Properties of DAGs

→ **Reachability**: Node A said to be reachable from node B, if there exists a directed path, that starts at node B and ends node A.
 (a walk that uses 1 node only once to traverse)

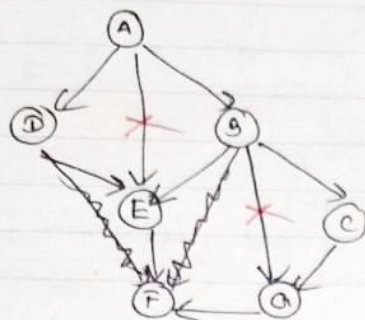
→ **Transitive Closure**: transitive closure is another graph that represent all directed and indirect connections between nodes in original graph.



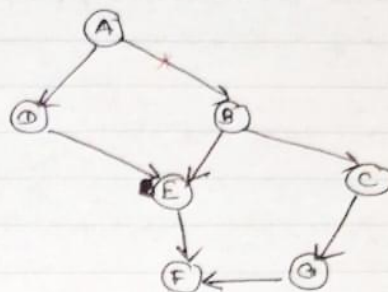


→ Transitive Reduction : A directed graph, that has only essential direct relationships between nodes. removing unnecessary directed edges.

EX :



* Original DAG.

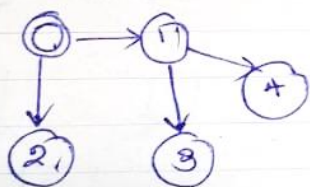


* Transitive Reduction.

* → Topological Ordering : DAG can be topologically sorted, that means we can linearly order the nodes in such a way that for all the edges, start the node of edges occurs earlier in the sequence. Used for scheduling and dependency resolution.

* DAG is a non-linear data structure.

[Coding assignment]

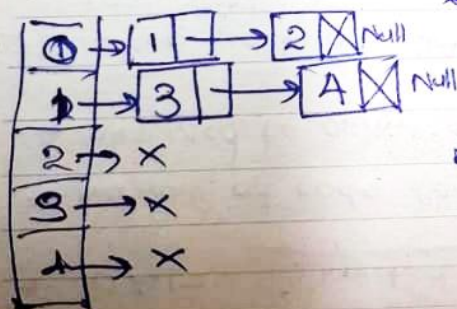


* DAGs can be represented (implemented as an) adjacency list.

* Adjacency list represent graph as a collection of lists.

* Adjacency list is an array or array list made from linkedlists.

* Each element is a separate linkedlist and linkedlists can contain nodes.



* Adjacency list use to store sparse graphs (many nodes low vertices)

Adjacency list[i] → Neighbour A → Neighbour B → Null.
(110) (11)

* Adjacency list can be implemented using 2 ways

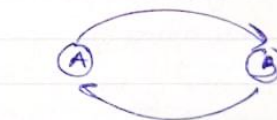
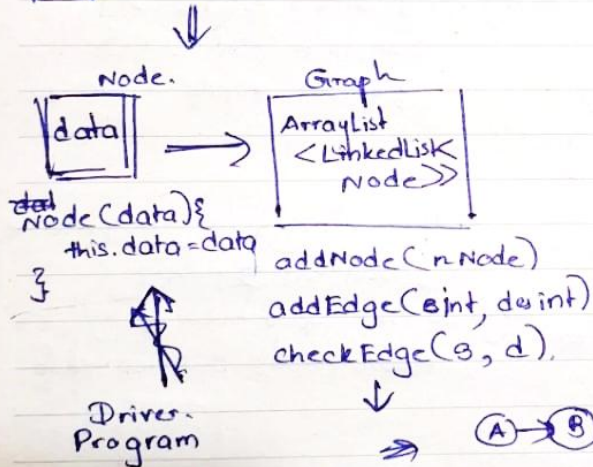
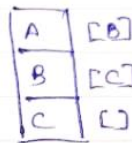
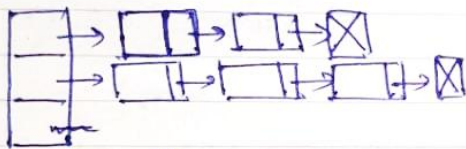
Array [LinkedList] based

(Slower due to pointer)

HashMap [Dynamic Array] based

(faster):

- good for traversal



Dangling nodes: links to page (Node) with no outgoing links.

AL Graph graph = new Graph();
graph.addNode(new Node('A'));
graph.addNode(new Node('B'));
graph.addEdge(0, 1);

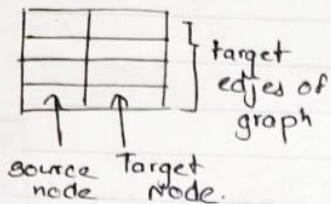
* can use singly linkedlist to implement DAG.

Assignment

input
csv file.

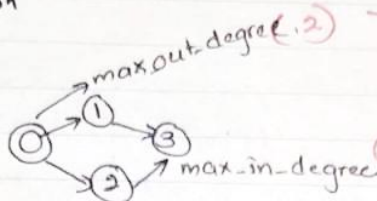
Program

```
is-dag: yes
max-in-degree: 2
max-out-degree: 2
pr-max:
pr-min:
```



Ex:

0	1
0	2
1	3
2	3



* graph can be a sparse graph
(~~to~~ many nodes, less edges).

$$|Edges| \ll |Vertices|^2$$

* can have self-edges (loops).

read csv

generate graph.

- using linked list
- using hash map

adjacency list

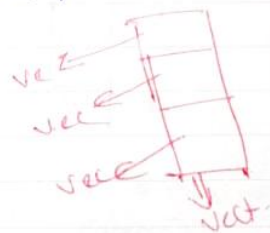
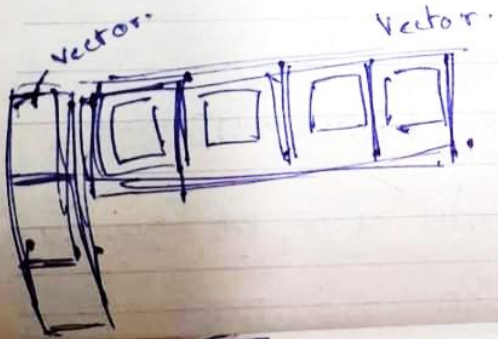
* create lib cargo new --lib
use dir :: fn-name;
* in same dir use mod.

let ans = match result {

OK(value) => value,

Err(err) => return Error(err.into());

let ans = result;



*** Page rank:** A method for computing a ranking for every webpage based on the graph of the web

- Page rank has applications in search, browsing and traffic estimation.

- A page has high rank if the sum of the ranks of its backlinks is high (backlist: inedges) (A, B are backlinks of C)

*** Definition of page rank:**

- let u be a page and F_u are pages that u points to and B_u are pages (nodes) that points to u .

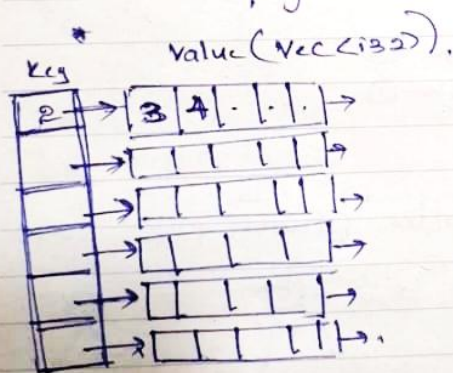
* $N_u = |F_u|$ (no. of links that from u)

* c - a factor used to normalize so that total rank of web pages are constant.

$$R(u) = c \sum_{v \in B_u} \frac{R(v)}{N(v)}$$

Simplified version of page rank.

* Rank of a page is divided among its forward links evenly to contribute, to rank of pages that it contribute to.

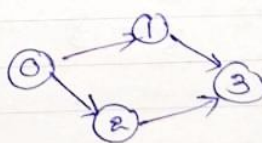


HashMap
[2, 3]
[2, 4]
↑ ↑
from to.

* $\text{max-out-degree} = \max(\text{size}(\text{vec}))$

* $\text{max-in-degree}:$

(0, 1) (0, 2) (1, 3) (2, 3)

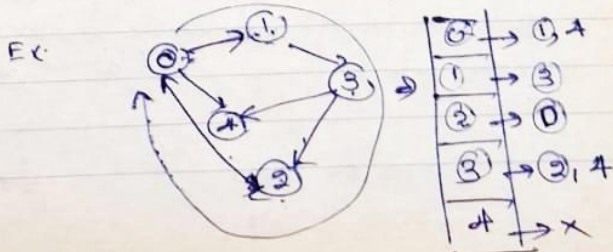


in-degree-count.

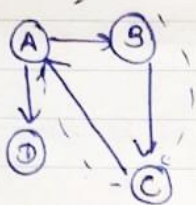
iterate each array and add counter.

*

if is a DAG, there should be a loop.



Detecting cycles in a graph



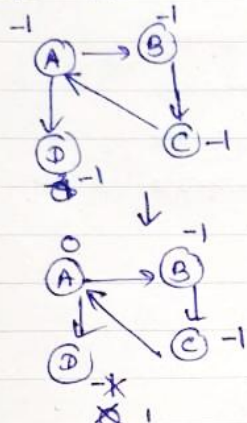
Flags

* -1 (unvisited)

* 0 (visited)

* 1 (visited & removed from stack)

start from A



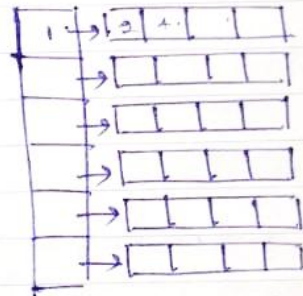
Parent Map

Vertex	Parent
A	-
D	A

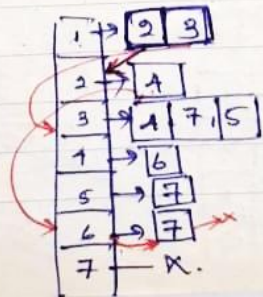
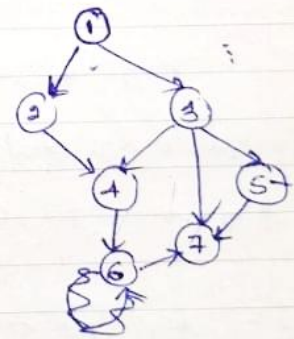
Visited Nodes

A
D ✓

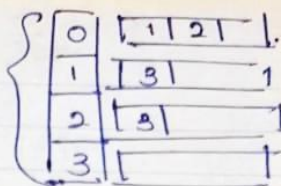
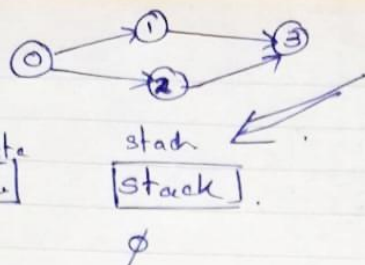
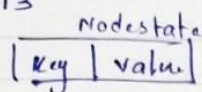
Key Value (vector)



- * get an element (Node)
- update visited Nodes List, add parent
- update flag (-1, 0, 1) as visited
- add to stack
- * get next element
- * add parent
- * update visited list
- * update flag.
- * check if there are can move forward if yes → add current to stack
- no: mark as 1 remove from stack.



0,1
0,2
1,3
2,3



iterate each key (0, 1, 2, 3)

* Page Rank: let graph G with vertices & edges $G(V, E)$ such that

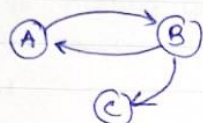
$$P(i) = \sum_{(j,i) \in E} \frac{P(j)}{q_j} \rightarrow \text{prestige of all the pages that are pointing towards } (i)$$

↑
prestige value of page i

↓
no. of outlets that they have

* The prestige (the importance of a page i (rank score) is the sum of page rank scores of all pages that point to it.

Ex:



$$P(A) = \frac{P(B)}{\text{no. of votes that B has}} = \frac{P(B)}{2} \text{ if } P(B) = 3$$

then $P(A) = 1.5$ recalculate $P(B)$ then.

$$P(B) = \frac{P(A)}{1} = 1.5$$

* What is markov chain? A stochastic process that satisfy markov property. Markov property explains that the probability distribution of next state depends only on current state - independent of previous history.

↓
sequence of random variables, each observation at a time gives a random outcome.

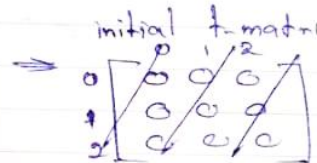
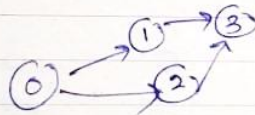
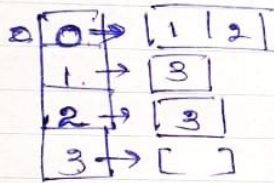
* What is a transition matrix? special type of square matrix used in markov chains. It defines the probabilities of moving from one node (state) to another in a single step.

* transition matrix should be row-stochastic : sum of all elements in a given row is equal to 1

$$M = \begin{bmatrix} 0.5 & 0.25 & 0.25 \\ 0 & 1 & 0 \\ 0.5 & 1 & 0.5 \end{bmatrix}$$

$$\begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 1 & 1.25 \\ 0.5 & 0 & 0.25 \\ 0.5 & 0 & 0.25 \\ 0 & 0 & 0.25 \end{bmatrix} \end{matrix}$$

* populating transition matrix with initial values - **4x4**



0/0.5/0.5.

$$\begin{matrix} & 0 & 1 & 2 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$\begin{bmatrix} 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0.25 & 0.25 & 0.25 & 0 \end{bmatrix}$$

* stack buffer overrun (memory allocation of bytes failed xxxxxxxx bytes).

* occurs when program writes data outside the boundaries of buffer allocated.

- * stack is a fast fixed sized memory use to manage variables & data.

* stack buffer overrun occurs when function attempt to write more data into a fixed size local variable.

* when function tries to return corrupted address points to a corrupted location

> recursions

> unsafe rut

> large local variables -

* Option 1 moving from DMatrix to Csr Matrix.

↓
Dense

↑
sparse



W

CSR matrix → use a compressed sparsed row storage to convert
 CSR matrix data should have to be passed as triplet in form
 (row, col, value)

CSR structure: $A = \begin{pmatrix} 0 & 0 & 5.0 & 0 \\ 1.0 & 0.0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 2.0 \end{pmatrix}$
 Size: $N_{rows} \times N_{columns}$
 Row pointers: $[0, 1, 3, 3, 4]$
 * keys to compression, store
 starting index of V & C arrays For each new row. $P(i+1) - P(i)$
 size always = $n+1$
 * Column indices array: (C) : store column index of every value
 in V array. (indices)
 Ex: $C = [2, 0, 1, 1, 3]$

* Values (V) (data) = stores actual non zero data elements. of
 matrix:

$V = [5, 1, 8, 4, 2]$

Row	$P(i)$	$P(i+1)$	start index	end index	value/col indices	interpret.
0	0	1	0	0	$V[0] = 5$ $C[0] = 2$	$A_{0,2} = 5$
1	1	3	1	2	$V[1] = 1$ $C[1] = 0$ $V[2] = 8$ $C[2] = 1$	$A_{1,0} = 1$ $A_{1,1} = 8$

$$r_i = (1-d) + d \sum_{p_i \in B_i} \frac{r_j}{L_j}$$

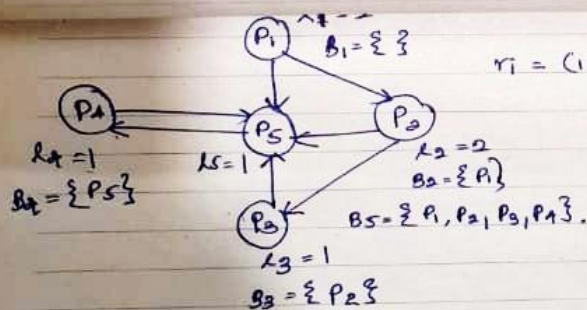
↓
damping
factor

P_i = Page

r_i = rank of P_i

L_i = outgoing links of P_i

B_i = pages link to P_i



$$r_i = (1-d) + d \sum_{B_i} \frac{r_j}{L_j}$$

(L_i = outdegree)

(B_i) = {in degree nodes}

$$r_1 = (1-d) + d \left[\frac{r_5}{L_1} \right] \therefore r_1 = (1-d) + d \times 0$$

$$r_2 = (1-d) + d \left[\frac{r_1}{L_2} \right] = (1-d) + d \times \frac{(1-d)}{2}$$

$$r_5 = (1-d) + d \left[\frac{r_1}{L_5} + \frac{r_2}{L_5} + \frac{r_3}{L_5} + \frac{r_4}{L_5} \right]$$